Pub-Sub systemen met message/service bus

Steden met ordercomponent

Groningen	0
Hengelo	0
Den Haag	0
Enschede	0
Heerlen	0
Mechelen	0

Leegmaken

Uw boekingen:

Trip naar:	Aantal	Prijs
Heerlen	1	€69.50
Groningen	1	€99.00
Totaal		**€168.50**

Annuleren Bevestig

City details +1 -1

Naam: Groningen

Provincie: Groningen

Highlights: Martinitoren,Grote Markt

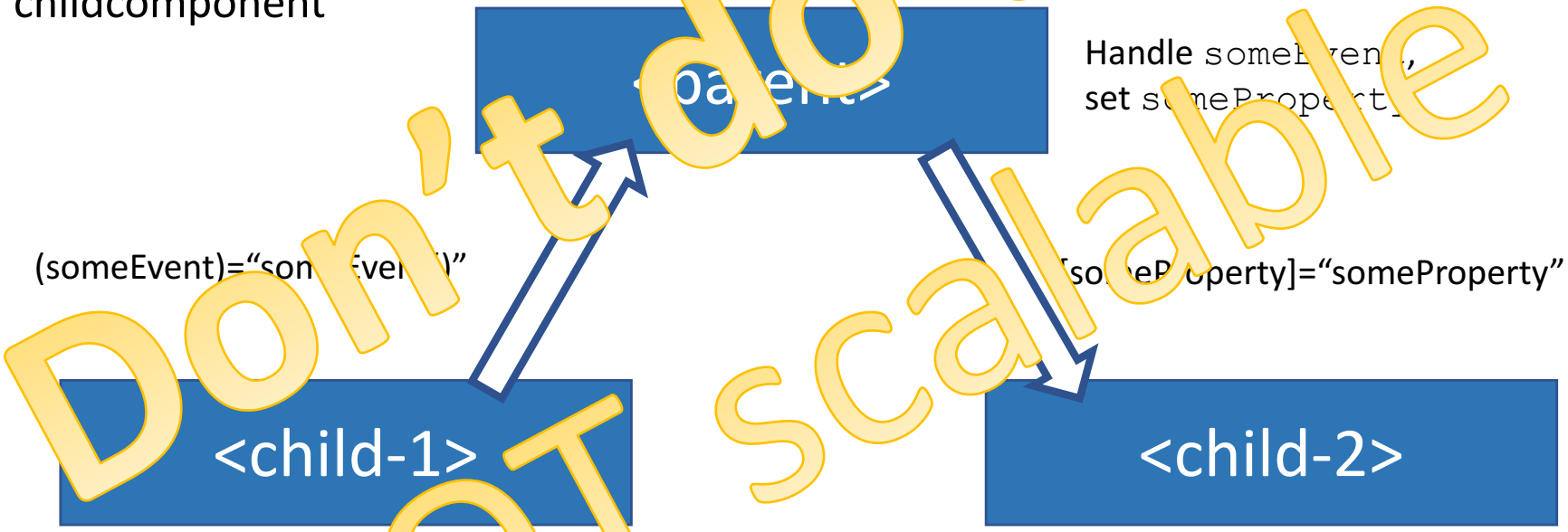
Prijs voor een weekendje weg: €99.00

Boek nu!

- Voorbeeld: https://github.com/PeterKassenaar/voorbeeldenAngular2/tree/master/oefeningen/303-pubsub-ordercomponent

Communicatie tussen siblings

- via `Output()` van een childcomponent, naar ee `@Input()` van andere childcomponent



Don't do this!

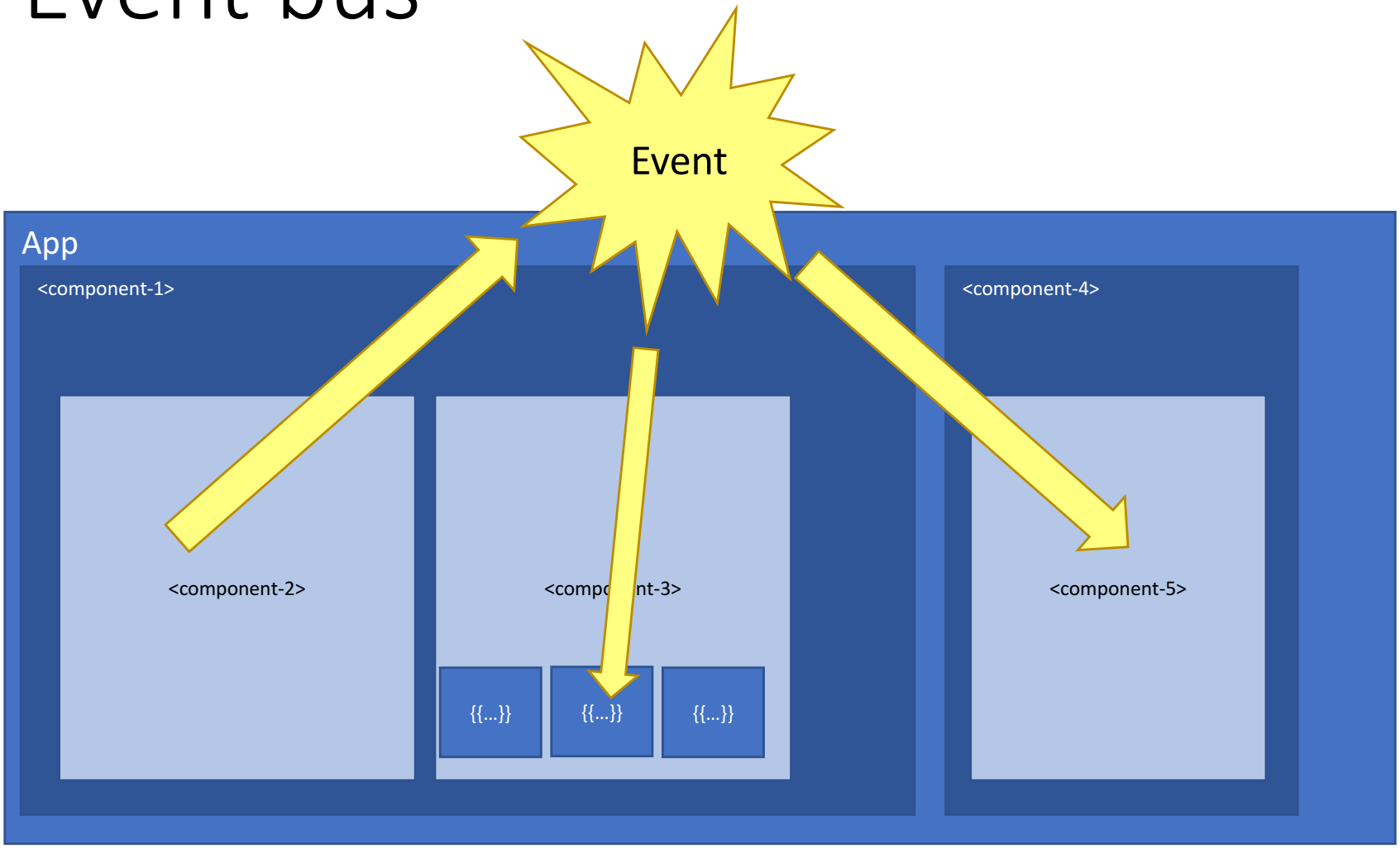
NOT scalable

Mooiere oplossing – Pub/Sub-systeem met Observables

- http://www.syntaxsuccess.com/viewarticle/pub-sub-in-angular-2.0

"Custom events,
gebruik een event bus"

Event bus

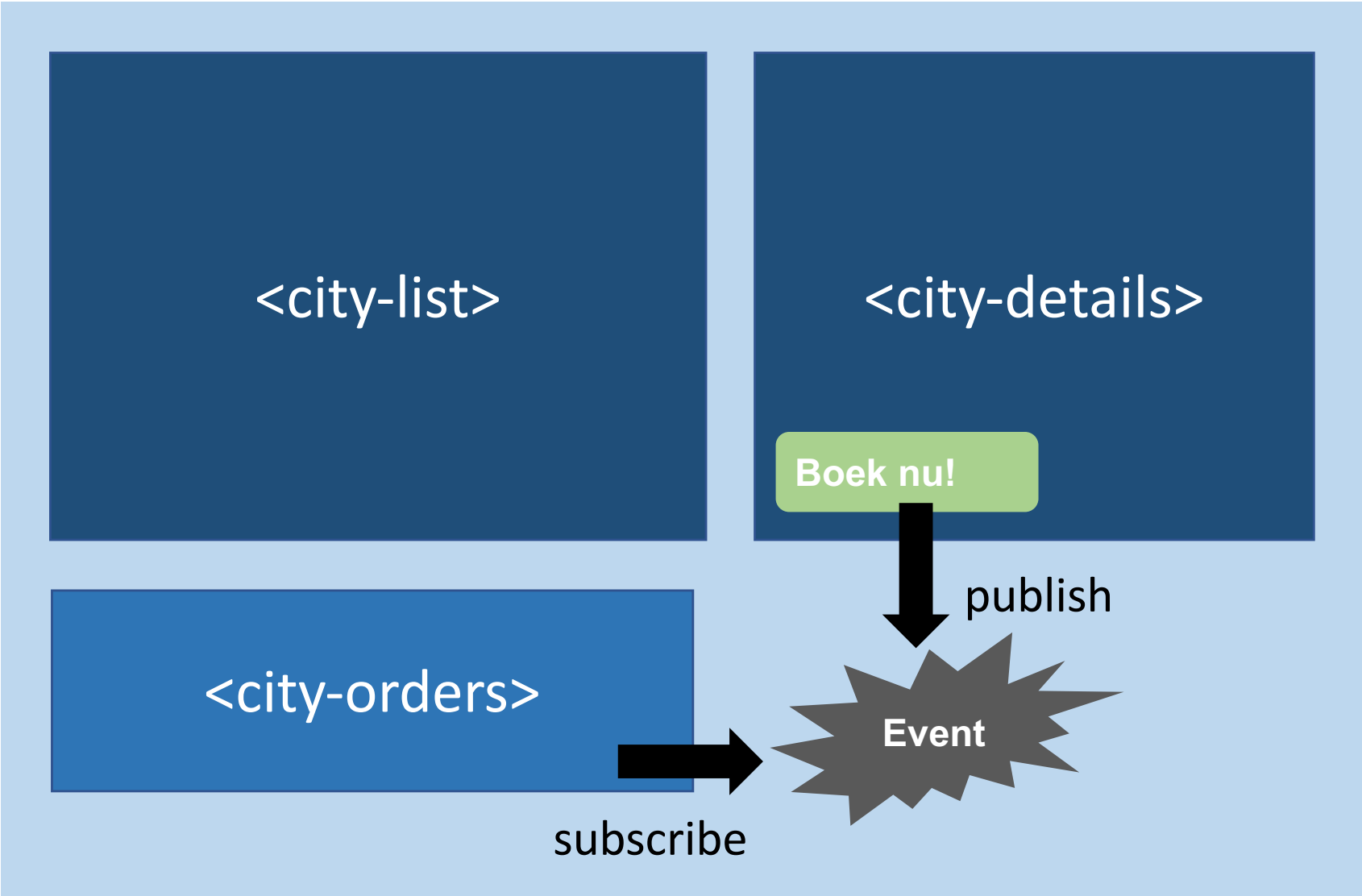

Event
App
<component-1>
<component-2>
<component-3>
{{...}}
{{...}}
{{...}}
<component-4>
<component-5>

Opties

Uit RxJs-bibliotheek, werken met:

- `EventEmitter()`
- `Observable()`
- `Observer()`
- `Subject()` (zowel `Observable` als `Observer`)

“Publish en Subscribe” – PubSub systeem


<city-list>
<city-details>
Boek nu!
publish
<city-orders>
Event
subscribe

PubSub-service maken

- Stap 1 – Publicatie service maken
- Stap 2 – ‘Producer’, of ‘Publish’ – component maken
- Stap 3 – subscriber-component maken, of toevoegen aan bestaande component.

1. OrderService

```
// order.service.ts
import {Subject} from "rxjs/Subject";
import {Injectable} from "@angular/core";
import {City} from "../model/city.model";

@Injectable()
export class OrderService {
   Stream:Subject<City>;

   constructor() {
      this.Stream = new Subject<City>();
   }
}
```

2. Producer component (‘boek nu’-knop)

In de HTML:

```
<h2>Prijs voor een weekendje weg:
{{ city.price | currency:'EUR':true:'1.2' }}
<button class="btn btn-lg btn-info"
   (click)="order(city)">Boek nu!</button>
</h2>
```

In de class:

```
// Order plaatsen. Event emitten voor deze stad.
// Dit gaan opvangen in city.orders.ts
order(city) {
   console.log(`Stedentripje geboekt voor: ${this.city.name});
   this.orderService.Stream.next(city);
}
```

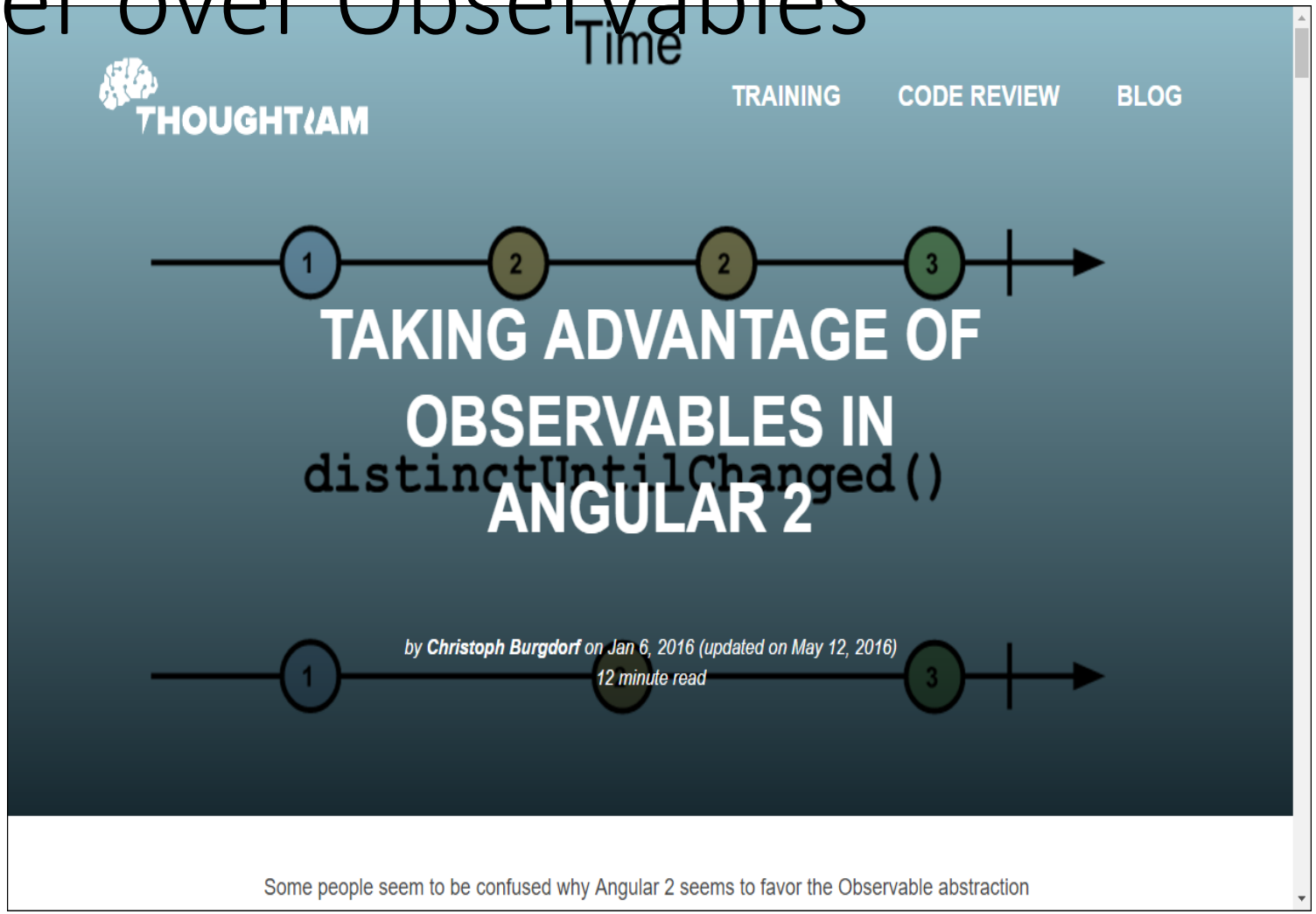
3. Subscriber component

```
//city.orders.ts - Een soort 'winkelmandje',
// bijhouden welke stedentripjes zijn geboekt.
import …

@Component({
   selector: 'city-orders',
   template: `
   <div *ngIf="currentOrders.length > 0">
      …
})

export class CityOrders {
   …
   ngOnInit() {
      this.orderService.Stream
         .subscribe(
            (city:City) => this.processOrder(city),
            (err)=>console.log('Error bij verwerken City-order'),
            ()=>console.log('Complete...')
         )
   }
`    …
}
```

Meer over Observables



http://blog.thoughtram.io/angular/2016/01/06/taking-advantage-of-observables-in-angular2.html

CORY RYLAN WORK CONTACT

My name is Cory Rylan, Senior Front End Engineer at Vintage Software and Angular Boot Camp instructor. I specialize in creating fast, scalable, and responsive web applications.

Follow @SplinterCode

Angular 2 Observable Data Services

Nov 17, 2015
Updated May 6, 2016 - 8 min read

Angular 2 brings many new concepts that can can improve our JavaScript applications. The first new concept to Angular is the use of Observables. Observables are a proposed feature for ES2016 (ES7). I wont go in depth into Observables but will just cover some of the high level concepts. If you want a introduction to Observables check out my screen cast.

INTRO TO RXJS OBSERVABLES AND ANGULAR 2

The rest of this post will cover more data and application state management in a Angular 2 application. At the time of this writing Angular is on version Beta 1. This post has been updated as of Beta 15. The syntax of how Observables and their

https://coryrylan.com/blog/angular-2-observable-data-services

Check out my Angular 2.0 article series

Home

Most Popular

Most Recent

Angular

React

Aurelia

JavaScript

NodeJS

MongoDB

Unit Testing

.Net

Q&A

All

Observables In Angular 2.0

Author: Torgeir Helgevold

Published: Wed Jan 06 2016

Viewed 3375 times

The RxJs community has presented the idea that any series of events can be modeled as one or many asynchronous or synchronous arrays. In the following post I want to explore this by modeling a series of different user inputs as Observables.

I am still learning about Observables and their potential, but I figured it would be interesting to implement a custom text editor, from scratch, using Observables to represent keyboard and mouse events.

Building a perfect text editor is not really the point here, but I want to see if there is any added value from looking at input sequences as Observables. The first step when building a text editor is identifying which input events to support. In my sample I have decided to focus on adding the ability to input and delete characters. Currently I have limited the input

http://www.syntaxsuccess.com/viewarticle/observables-in-angular-2.0